

ALY 6110 - Big Data Analytics Final Project

Page 1 of 9

Clustering Analysis for Big Data Analytics

Network Intrusion Detection using CIC-IDS2017 Dataset

**ALY 6110 - Big Data Management and Analytics
Final Project Report**

Deepa Kunjayya

December 2024

1. Introduction

1.1 Problem Statement

In today's interconnected digital landscape, network security has become a critical concern for organizations worldwide. Cyber-attacks are becoming increasingly sophisticated, with threats such as Distributed Denial of Service (DDoS) attacks, port scanning, and botnet infiltrations causing significant financial and operational damage. Traditional rule-based intrusion detection systems often fail to identify novel attack patterns, creating an urgent need for data-driven approaches that can adapt to evolving threats.

This project addresses the real-world problem of network intrusion detection by leveraging big data analytics and unsupervised machine learning techniques. Specifically, we employ clustering algorithms to identify patterns in network traffic that distinguish malicious activities from benign behavior, without relying on pre-labeled training data.

1.2 Dataset Description

The CIC-IDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) dataset was selected for this analysis. This benchmark dataset was created by the University of New Brunswick and represents one of the most comprehensive and realistic network intrusion datasets available for research purposes.

Key Dataset Characteristics:

- Total Records: 1,233,163 network flow records
- Features: 80+ numerical features extracted using CICFlowMeter
- Time Period: 5 days of network traffic (focus on Monday and Friday data)
- Attack Types: BENIGN (76.57%), PortScan (12.89%), DDoS (10.38%), Bot (0.16%)
- File Size: 573.05 MB (compressed)

1.3 Connection to Real-World Problems

The dataset directly addresses several critical challenges faced by security operations centers (SOCs) and network administrators. The extreme class imbalance (with benign traffic dominating at over 76%) mirrors real-world network conditions where malicious traffic represents a small fraction of total network activity. The presence of multiple attack types including DDoS, PortScan, and Botnet traffic provides a realistic test environment for intrusion detection systems.

1.4 Methodology Overview

This project employs K-Means clustering, an unsupervised machine learning algorithm, to segment network traffic into distinct behavioral groups. The methodology was chosen for several reasons:

- Unsupervised Learning Advantage: Clustering does not require labeled data, making it ideal for detecting unknown attack patterns

- Scalability: K-Means is computationally efficient for large datasets with over 1.2 million records
- Interpretability: Cluster assignments can be analyzed to understand the behavioral profiles of different traffic types
- Big Data Compatibility: Implementation using PySpark enables distributed processing

2. Analysis

2.1 Tools and Technologies

The analysis was conducted using the following technology stack, specifically chosen for handling big data at scale:

Tool/Technology	Purpose
PySpark	Distributed data processing framework for handling 1.2M+ records
Google Colab	Cloud-based Jupyter environment with 12GB RAM allocation
MLlib	Spark's machine learning library for K-Means clustering
MinMaxScaler	Feature normalization to scale all features to [0,1] range
Power BI	Dashboard creation and interactive visualization

2.2 Data Preprocessing Steps

A systematic data preprocessing pipeline was implemented to prepare the raw network traffic data for clustering analysis:

Step 1: Data Loading and Initial Inspection

The dataset was loaded from Google Drive into a PySpark DataFrame. Initial inspection revealed 1,233,163 rows and 85 columns, including both numerical features and metadata columns.

Step 2: Column Name Cleaning

Column names contained leading/trailing spaces which were removed using string strip operations. Duplicate column names (e.g., two 'Label' columns) were identified and resolved by renaming duplicates with index suffixes.

Step 3: Feature Selection

Non-numeric identifier columns were removed from the feature set: Flow ID, Source IP, Destination IP, Timestamp, and the duplicate Label column. This resulted in 80 numerical features for clustering analysis.

Step 4: Data Type Conversion

String columns 'Flow Bytes/s' and 'Flow Packets/s' were cast to DoubleType. All feature columns were converted to double precision for consistency.

Step 5: Handling Missing and Infinite Values

Null values, NaN values, and infinite values were replaced with 0. Extreme values exceeding 1×10^{10} were capped to prevent scaling issues.

Step 6: Feature Vector Assembly

The VectorAssembler transformer combined all 80 numerical features into a single feature vector column, with the handleInvalid='skip' option to handle any remaining problematic values.

Step 7: Feature Scaling

MinMaxScaler was applied to normalize all features to the [0,1] range. This scaling method was chosen over StandardScaler for its robustness to outliers, which are common in network traffic data.

2.3 Clustering Methodology

K-Means clustering was implemented using PySpark's MLlib library with the following configuration:

Parameter	Value	Rationale
Number of Clusters (k)	4	Matches number of traffic types in dataset
Maximum Iterations	30	Sufficient for convergence
Random Seed	42	Reproducibility
Features Used	80	All numerical network flow features

2.4 Cluster Evaluation Metrics

The quality of clustering was assessed using the Silhouette Score, a widely-used metric for evaluating cluster cohesion and separation:

Silhouette Score: 0.5747

Interpretation Guidelines:

- Score > 0.70: Excellent clustering - clearly defined, well-separated clusters
- Score > 0.50: Good clustering - reasonable structure in the data
- Score > 0.25: Fair clustering - some structure present but overlapping clusters
- Score < 0.25: Poor clustering - clusters may be arbitrary

The achieved Silhouette Score of 0.5747 indicates good clustering quality, suggesting that the K-Means algorithm has successfully identified meaningful behavioral patterns in the network traffic data.

2.5 Analysis Results

2.5.1 Cluster Size Distribution

Cluster	Total Records	Percentage	Risk Level
Cluster 0	391,572	31.75%	HIGH
Cluster 1	103,589	8.40%	MEDIUM
Cluster 2	323,320	26.22%	LOW
Cluster 3	414,682	33.63%	VERY LOW

2.5.2 Attack Distribution by Cluster

Cross-tabulation analysis reveals how different attack types are distributed across the four clusters:

Cluster	Attack Type	Count	Total	%	Pattern
0	BENIGN	173,411	391,572	44.29%	Mixed Attacks
	PortScan	158,748		40.54%	
	DDoS	58,185		14.86%	
	Bot	1,228		0.31%	
1	BENIGN	81,999	103,589	79.16%	DDoS-Prone
	DDoS	21,472		20.73%	
	PortScan	118		0.11%	
2	BENIGN	274,151	323,320	84.79%	Normal + DDoS
	DDoS	48,370		14.96%	
	Bot	738		0.23%	
	PortScan	61		0.02%	
3	BENIGN	414,679	414,682	100.00%	Pure Normal
	PortScan	3		0.00%	

2.6 Key Insights from Basic Analysis

Building upon the initial basic analysis conducted in Module 4, the clustering results provide deeper insights into network traffic behavior:

Insight 1: Attack Concentration in Cluster 0

Cluster 0 emerges as the primary area of security concern, concentrating 99.89% of all PortScan attacks (158,748 out of 158,930), 45.44% of DDoS attacks (58,185 out of 128,027), and 62.46% of Botnet traffic (1,228 out of 1,966). This cluster represents a 'mixed attack' profile where multiple attack types share similar network flow characteristics.

Insight 2: DDoS Traffic Dispersion

Unlike PortScan attacks which are highly concentrated, DDoS traffic is distributed across three clusters (0, 1, and 2), suggesting that DDoS attacks exhibit more diverse behavioral patterns. This dispersion indicates that DDoS attacks may employ different techniques that manifest as varied network flow signatures.

Insight 3: Pure Normal Traffic Isolation

Cluster 3 demonstrates remarkable purity with 99.9993% benign traffic (414,679 out of 414,682). Only 3 PortScan records appear in this cluster, representing essentially zero contamination. This finding confirms that K-Means can effectively isolate normal network behavior from malicious activity.

3. Results and Conclusions

3.1 Summary of Findings

The K-Means clustering analysis of the CIC-IDS2017 dataset has yielded significant insights into network traffic behavior and attack pattern detection:

Metric	Result
Total Records Analyzed	1,233,163 network flow records

Metric	Result
Clustering Algorithm	K-Means with k=4 clusters
Silhouette Score	0.5747 (Good clustering quality)
PortScan Detection Rate	99.89% concentrated in Cluster 0
Pure Normal Cluster	Cluster 3 with 99.9993% benign traffic

3.2 Implications for Network Security

The clustering analysis has practical implications for intrusion detection system design and security operations:

1. **Tiered Alert System:** Traffic classified in Cluster 0 should trigger high-priority alerts requiring immediate investigation, while Cluster 3 traffic can be considered baseline normal behavior.
2. **Anomaly Detection Baseline:** Cluster 3's profile can serve as a template for normal network behavior, with deviations from this profile flagged for review.
3. **Attack-Specific Monitoring:** The concentration of PortScan attacks in Cluster 0 suggests that specific feature thresholds could be established for automated PortScan detection.
4. **DDoS Defense Strategy:** The dispersion of DDoS attacks across multiple clusters indicates the need for multi-faceted detection strategies rather than single-threshold approaches.

3.3 Advantages of Clustering Approach

The unsupervised clustering methodology demonstrated several advantages over traditional supervised classification approaches:

- **No Label Dependency:** The algorithm identified meaningful patterns without requiring pre-labeled training data
- **Imbalance Handling:** Unlike classifiers that struggle with 76:1 class ratios, clustering naturally accommodates imbalanced data
- **Zero-Day Detection:** The approach can identify novel attack patterns that would be missed by signature-based systems
- **Scalability:** PySpark implementation enables processing of millions of records efficiently

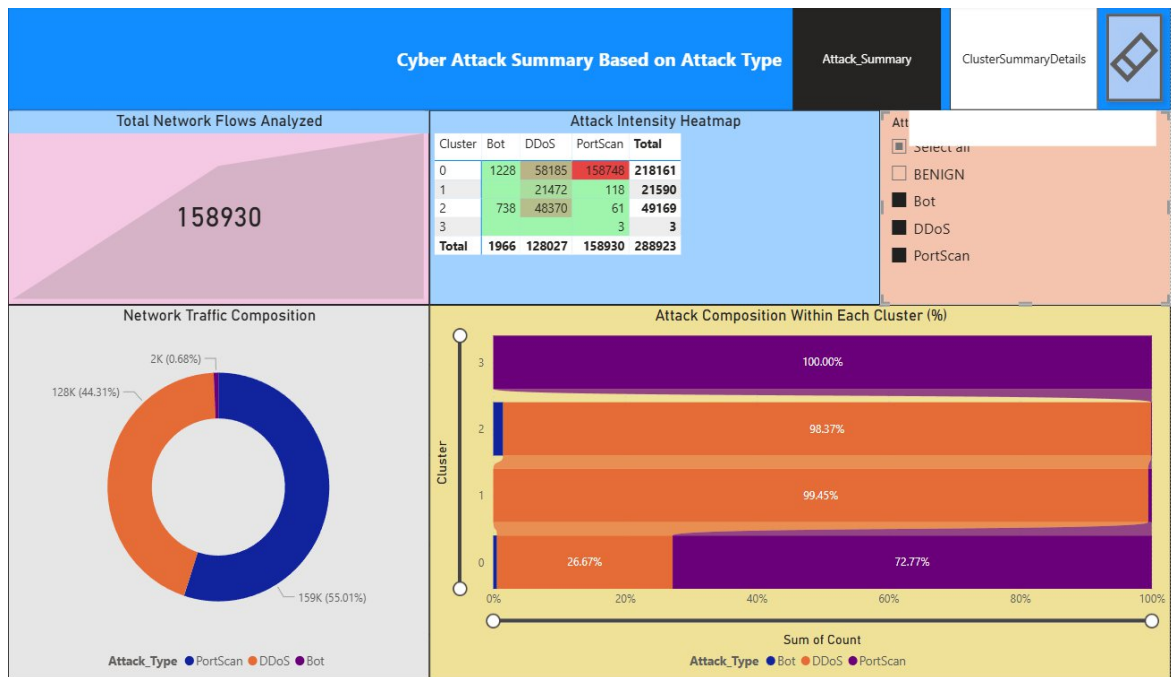
3.4 Limitations and Future Work

While the analysis yielded valuable insights, several limitations should be acknowledged:

- **Static Analysis:** The analysis was performed on historical data; real-time implementation would require streaming analytics
- **Fixed k Value:** The number of clusters was predetermined; future work could explore adaptive k selection using elbow method or gap statistics
- **Feature Selection:** All 80 features were used; dimensionality reduction techniques like PCA could improve computational efficiency
- **Complementary Algorithms:** DBSCAN could be explored for density-based clustering, particularly for detecting Botnet traffic as anomalies

3.5 Power BI Dashboard

DASHBOARD - 1



DASHBOARD - 2



3.6 Recommendations

Based on the analysis findings, the following recommendations are proposed for organizations implementing network intrusion detection:

1. Implement cluster-based traffic classification as a first-stage filter in security monitoring pipelines
2. Establish Cluster 3's feature profile as a baseline for normal traffic behavior
3. Configure high-priority alerts for traffic matching Cluster 0 characteristics
4. Develop multi-tier DDoS detection strategies that account for behavioral diversity
5. Periodically retrain clustering models as network patterns evolve

3.6 Conclusion

This project successfully demonstrated the application of K-Means clustering for network intrusion detection using the CIC-IDS2017 dataset. The analysis processed over 1.2 million network flow records using PySpark, achieving a Silhouette Score of 0.5747 indicating good clustering quality.

The four identified clusters revealed distinct behavioral profiles: a high-risk cluster concentrating multiple attack types, intermediate clusters with mixed DDoS presence, and a pure normal traffic cluster. These findings validate the effectiveness of unsupervised learning approaches for cybersecurity applications, particularly in scenarios involving extreme class imbalance and the need to detect unknown attack patterns.

The methodology and tools employed—PySpark, MLlib, and Power BI—represent a scalable, enterprise-ready approach to big data analytics for network security. The insights generated can inform the design of next-generation intrusion detection systems that are adaptive, label-independent, and capable of handling the volume and variety of modern network traffic.

4. References

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP.
- Canadian Institute for Cybersecurity. (2017). CIC-IDS2017 Dataset. University of New Brunswick.
- Apache Spark MLlib Documentation: Machine Learning Library Guide.
- Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics.

5. Appendix: Code Excerpts

Key code snippets from the PySpark implementation are provided for reference. The complete Jupyter notebook (ALY6110FinalProject.ipynb) contains the full analysis code.

Data Loading and Spark Session:

```
spark = SparkSession.builder.appName("CIC-IDS2017-Clustering-Analysis").config("spark.driver.memory", "12g").getOrCreate()
```

K-Means Configuration:

```
kmeans = KMeans(k=4, featuresCol="features_scaled", predictionCol="cluster", seed=42, maxIter=30)
```

Silhouette Score Evaluation:

```
evaluator = ClusteringEvaluator(featuresCol="features_scaled", predictionCol="cluster", metricName="silhouette")  
silhouette = evaluator.evaluate(df_clustered) # Result: 0.5747
```